

实验3报告：基于 Transformer 的中文诗歌生成

1. 实验目的

- 在不依赖大语言模型微调（不使用 LoRA/QLoRA/外部 LLM）的条件下，实现可训练的中文诗歌生成模型。
- 设计并验证“格式约束 + 押韵约束 + 可选平仄约束”的生成策略。
- 建立多维自动评估体系，对生成质量进行量化分析。

2. 实验环境

2.1 软件环境

- Python 3.x
- PyTorch（GPU 训练，支持 AMP 混合精度）
- 可选：pypinyin（用于平仄分析）

2.2 实验数据

- 训练数据：../data/tang.npz，共有57,580首唐诗，使用NPZ压缩格式，包含数据矩阵和词汇映射
- 任务形式：字符级语言建模 + 约束解码生成绝句

3. 实验原理

3.1 模型结构

本实验使用字符级因果 Transformer，主要由以下部分组成：

- Token Embedding
- 绝对位置 Embedding
- 循环句位 Embedding (pattern_cycle)
- 多层 Transformer Encoder（通过因果 mask 实现自回归）
- 线性输出头（与输入嵌入权重共享）

结构概览（当前配置）：

PoetryTransformer (13.89M 参数)

- 词嵌入层 (8,293 词汇量 → 384 hidden)
- 位置编码层
 - 绝对位置嵌入 (max_seq_len=128)
 - 循环句位嵌入 (pattern_cycle=8)
- 6层 Transformer 解码式编码块 (因果 mask)
 - 多头自注意力机制 (6 heads, 384 hidden)
 - 前馈神经网络 (1,536 dim)
 - 残差连接 + LayerNorm
- 最终 LayerNorm
- 语言模型头 (384 → 8,293, 且与词嵌入权重共享)

3.2 约束生成机制

- 格式约束：按 `line_length` 与 `num_lines` 控制每句长度与句末标点。
- 押韵约束：在指定行（默认第 1/2/4 句）限制候选字到同韵簇。
- 藏头约束：每句句首可强制为给定藏头字。
- 采样策略：`temperature + top-k + top-p + repetition penalty`。

3.3 核心代码实现

训练阶段（梯度累积 + AMP + 学习率调度）：

```
criterion = nn.CrossEntropyLoss(ignore_index=pad_idx) if pad_idx is not None
else nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=lr, weight_decay=weight_decay)

amp_enabled = bool(use_amp and device.startswith("cuda"))
scaler = torch.amp.GradScaler("cuda", enabled=amp_enabled)

updates_per_epoch = math.ceil(len(dataloader) / max(1, grad_accum_steps))
total_updates = max(1, updates_per_epoch * epochs)
warmup_steps = int(total_updates * max(0.0, warmup_ratio))
scheduler = _build_scheduler(
    optimizer=optimizer,
    scheduler_type=scheduler_type,
    total_steps=total_updates,
    warmup_steps=warmup_steps,
    min_lr_ratio=min_lr_ratio,
)
```

生成阶段（结构约束 + 押韵约束）：

```
if use_constraints and pos_in_line <= line_length:
    logits = _block_inner_punctuations(logits, word2ix=word2ix)

if use_rhyme_constraint and is_line_end_char and line_no in rhyme_lines:
    if selected_rhyme_char is None:
        logits = _constrain_rhyme_seed(logits, word2ix=word2ix,
rhyme_lexicon=rhyme_lexicon)
    else:
        logits = _constrain_rhyme_follow(
            logits,
            word2ix=word2ix,
            target_rhyme_char=selected_rhyme_char,
            rhyme_lexicon=rhyme_lexicon,
        )
```

3.4 代码中的特殊设计详解

本项目的生成思路不是单纯依赖随机采样，而是将“模型生成能力”和“诗体规则控制”结合起来，主要体现在以下几点：

- 在押韵方面，系统会先从语料中归纳常见韵脚关系，再在生成关键位置保持韵脚一致，因此能在不引入复杂外部词典的前提下获得较稳定的押韵效果。
- 在结构方面，生成过程会控制句长与标点位置，使输出更贴近绝句的基本格式，减少结构性错误。
- 在约束协同方面，整体采用“规则先约束、再采样补充多样性”的策略，在可控性与自然度之间做平衡。
- 在韵律方面，平仄能力设计为可选模块；当外部依赖不可用时可自动回退，保证代码在不同环境下都能稳定运行。
- 在模型表达方面，除常规位置信息外还引入了句位周期信息，以更好适应诗歌中重复出现的行文节奏。
- 在实验流程方面，训练、生成与评估形成闭环，能够从格式、押韵、重复度等多个维度观察模型效果。

整体来看，这些设计让系统既具有一定诗体可控性，也保留了生成式模型应有的灵活性。

3.5 当前架构与创新点

当前系统采用“**字符级 Transformer + 约束解码 + 自动评估**”的三段式架构：

- **模型层**：使用字符级自回归 Transformer，包含 token embedding、绝对位置 embedding 与循环句位 embedding，并采用输入输出权重共享。
- **训练层**：采用标准语言模型训练流程，支持 AMP、梯度累积、梯度裁剪、学习率调度与断点续训。
- **推理与评估层**：生成时使用结构化约束解码，评估时从格式、押韵、平仄、重复度与困惑度等维度综合分析质量。

在此基础上，本项目的主要创新点包括：

- **数据驱动的押韵建模**：不依赖手工硬编码韵书，而是从语料自动归纳韵脚关系，提升可迁移性。
- **“硬约束 + 软采样”协同生成**：先保证诗体结构与韵律要求，再保留一定采样随机性，兼顾可控性与自然度。
- **句位周期先验注入**：通过循环句位信息增强模型对诗歌节奏与行结构的建模能力。
- **可降级的平仄模块**：平仄相关能力为可选组件，缺少外部依赖时可自动回退，不影响主流程。
- **训练-生成-评估闭环**：不仅展示样例，还可通过统一指标体系持续比较不同配置与策略效果。

4. 训练配置与流程

4.1 主要训练配置

- `batch_size=16`
- `grad_accum_steps=2`
- `d_model=384, nhead=6, num_layers=6`
- `dim_feedforward=1536`
- `lr=1e-4, weight_decay=0.01`
- `scheduler=cosine, warmup_ratio=0.08`
- `max_grad_norm=0.5`
- `use_amp=True`

4.2 训练日志概览 (`log/history.csv`)

- 第 1 轮：`train_loss=34.841935, ppl=inf`
- 第 30 轮：`train_loss=4.927761, ppl=138.070038`

损失和困惑度持续下降，说明模型在训练语料上的拟合能力显著提升。损失相比较初期大幅下降，后期趋于平稳，符合预期的训练曲线趋势。

5. 实验结果

5.1 批量评估结果 (`eval_results.jsonl`, 20 条样本)

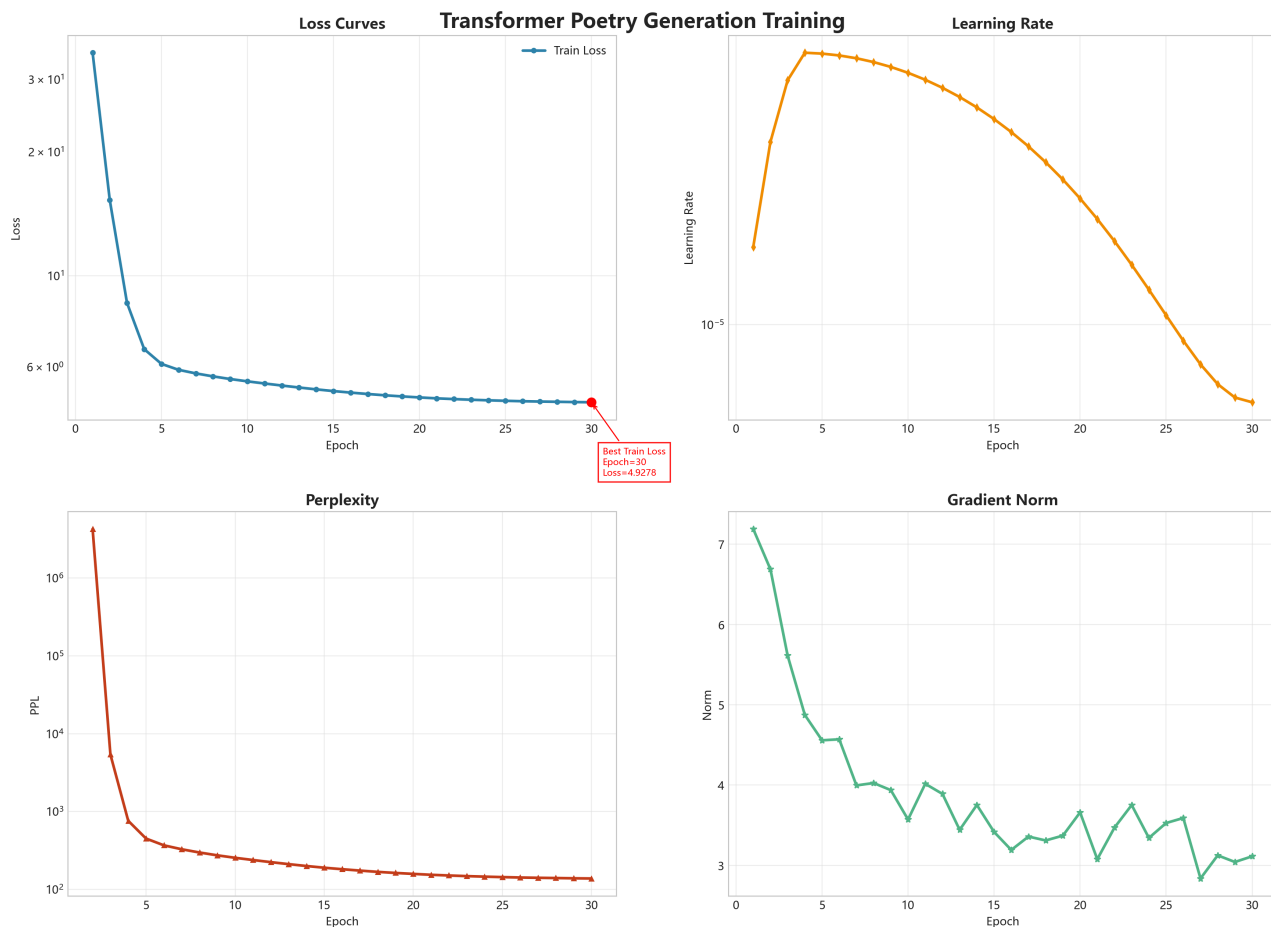
- `format_score` 平均值: 1.0000
- `rhyme_score` 平均值: 1.0000
- `zeqi_pingshou_score` 平均值: 0.5375
- `distinct1` 平均值: 0.9172
- `repetition2` 平均值: 0.0331
- `ppl` 平均值: 59.7442
- `quality_score` 平均值: 0.8157

最优样例 (按 `quality_score`):

- `prompt`: 碧涧苍松五粒稀
- `quality_score`: 0.8515
- `生成结果`: 碧涧苍松五粒稀，清风冷落白云飞。夜中玉殿寒露湿，天上新泉月初晴。

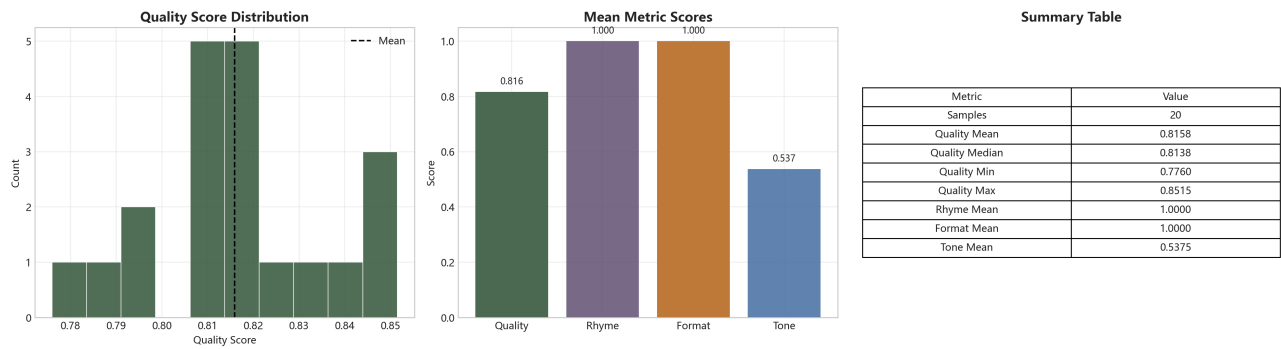
5.2 可视化结果

- 训练过程面板:



- 可见一开始损失较高，困惑度为无穷大，随着训练进行，损失迅速下降，困惑度也显著降低，表明模型在逐渐学会拟合训练数据。

- 评估质量分布与指标统计：



- 说明在当前评价指标体系下，生成质量整体较好，且格式与押韵约束得分稳定在满分水平，平仄得分则存在一定提升空间。

5.3 续写示例

```
(mineru_env) PS E:\hw\deep_learning\src_to_submit\exp3> python main.py --mode generate --ckpt ./checkpoints/poetry_transformer.pth --start_words 湖光秋月两相和
湖光秋月两相和，草木高低寒水流。白衣飞来碧玉树，黄金井上红莲楼。
```

- 可见模型能够较好地续写给定开头，保持诗体格式与韵律要求，且生成内容具有一定的语义连贯性和诗意表达。

5.4 藏头诗生成

```
(mineru_env) PS E:\hw\deep_learning\src_to_submit\exp3> python main.py --mode generate --ckpt ./checkpoints/poetry_transformer.pth --acrostic_text 春夏秋冬 --num_lines 4
春风吹玉塞金簪，夏日三千里相传。秋色天河水已远，冬声初开九流泉。
(mineru_env) PS E:\hw\deep_learning\src_to_submit\exp3> python main.py --mode generate --ckpt ./checkpoints/poetry_transformer.pth --acrostic_text 梅兰竹菊 --num_lines 4
梅花有白石头落，兰江半夜垂杨柳。竹叶风流清月明，菊花带露浓香冷。
(mineru_env) PS E:\hw\deep_learning\src_to_submit\exp3> python main.py --mode generate --ckpt ./checkpoints/poetry_transformer.pth --acrostic_text 春和景明 --num_lines 4
春风吹长水边行，和光照海带天明。景景何人更相送，明日出没无时情。
```

- 在多个藏头字的约束下，模型仍能生成符合格式与韵律要求的诗歌，且藏头字正确出现在每句开头，展示了较强的约束生成能力。

6. 实验总结

- 在非 LLM 微调约束下，实验成功实现了可训练的字符级 Transformer 诗歌生成系统。
- 结构化约束策略有效：格式与押韵指标均达到较高稳定性（平均 1.0）。
- 当前主要改进空间在平仄质量与语义连贯性，可进一步结合更精细的韵律词典、语义约束或更大规模预训练语料优化质量上限。